Darren Lee
danalee@ucsc.edu
4/18/2021

CSE 13S Spring 2021
Assignment 3: Sorting: Putting your affairs in order
Design Document

Pre-lab Part 1

1. How many rounds of swapping will need to sort the numbers 8,22,7,9,31,5,13 in ascending order using Bubble Sort?

It will take 5 rounds of swapping to put the numbers in ascending order using Bubble Sort.
After the first round: 8,7,9,22,5,13,31
After the second round: 7,8,9,5,13,22,31
After the third round: 7,8,5,9,13,22,31
After the fourth round: 7,5,8,9,13,22,31
After the fifth round: 5,7,8,9,13,22,31
Technically, there is a sixth pass in which the numbers are compared again (n=7 and n-1=6 passes), but since the numbers are already in order there is no more swapping after the fifth round.

2. How many comparisons can we expect to see in the worse case scenario for Bubble Sort? Hint:make a list of numbers and attempt to sort them using Bubble Sort.

According to https://www.geeksforgeeks.org/bubble-sort/, Bubble Sort has a worst case time complexity of $O(n^2)$, which occurs when the list is reverse sorted. For example, (5,4,3,2,1) would take 20 comparisons when using Bubble Sort, which is n^2-n for n=5. However, since n^2 is the dominating term and the -n does not make a large difference as the number of terms grows larger, the worst case time complexity is $O(n^2)$.

Pre-lab Part 2

1. The worst time complexity for Shell Sort depends on the sequence of gaps. Investigate why this is the case. How can you improve the time complexity of this sort by changing the gapsize? Cite any sources you used.

Shell Sort's worst time complexity depends on the sequence of gaps because that determines how many comparisons are made. Since we are using the Pratt sequence, the worst time complexity will be $O(n\log^2 n)$, but using the Knuth sequence with the formula $(3^K-1)/2$ would have a worst time complexity of $O(n^{3/2})$. I used https://www.codesdope.com/blog/article/shell-sort/ and https://en.wikipedia.org/wiki/Shellsort#Gap_sequences to answer this question.

Pre-lab Part 3

1. Quicksort, with a worst case time complexity of $O(n^2)$, doesn't seem to live up to its name. Investigate and explain why Quicksort isn't doomed by its worst case scenario. Make sure to cite any sources you use.

The worst cases of Quicksort occur when the array is sorted from least to greatest and the leftmost element is chosen as the pivot and when the array is sorted from greatest to least and the rightmost element is chosen. The array will be split into two very unbalanced sub-arrays, one with length 1 and one with length n-1. There are ways to avoid these scenarios such as picking a pivot from the middle of the array, selecting random pivots, or taking the median of 3 pivots. On average, the time complexity of Quicksort is O(nlogn). It performs well even with large arrays and is considered one of the best sorts regarding efficiency. I used https://www.baeldung.com/cs/quicksort-time-complexity-worst-case to answer this question.

Pre-lab Part 4

1. Explain how you plan on keeping track of the number of moves and comparisons since each sort will reside within its own file.

I plan to use global variables in my sorting files and then using external variables in the sorting test harness file to keep track of the number of moves and comparisons for each sort. For each comparison, I can increment the variable that counts comparisons, and for each movement in the array, I can increment the variable that counts moves. For quicksort, I can create a helper function that increments a global variable whenever a comparison is made.

Sources:

1. https://www.geeksforgeeks.org/bubble-sort/
2. https://www.codesdope.com/blog/article/shell-sort/
3. https://en.wikipedia.org/wiki/Shellsort#Gap_sequences
4. https://www.baeldung.com/cs/quicksort-time-complexity-worst-case

---

Purpose

The purpose of this program is to implement bubble sort, shell short, and two quicksorts. The program uses abstract data types, a stack for the first quicksort and a queue for the second quicksort. This lab will allow me to compare the sorting algorithms and better understand their implementations.

Layout/Structure

- The header files bubble.h, shell.h, and quick.h have the interface for the sorts.
- Sorting.c contains a test harness for the sorting algorithms which creates an array of random elements to be sorted.
- I will keep track of the sizeof the array, the number of moves required for each sort, and the number of comparisons required for each sort. I will also keep track of the maximum stack and queue sizes for the quicksorts.
- Bubble.c implements bubble sort, shell.c implements shell sort, quick.c implements both iterative quicksorts, stack.c implements the stack abstract data type, and queue.c implements the queue abstract data type.

Functionality

- Bubble sort swaps a pair of items in an array if one is smaller than the other. The largest element reaches the top of the array. The time complexity in the worst case is $O(n^2)$.
- Shell sort pairs up elements that have a large gap between them. Using gaps in a gap sequence, it swaps pairs if they are in the wrong order and starts with distant elements. The expected time complexity depends on the gap sequence. I will use the Pratt sequence (2^p3^q).
- Quicksort partitions arrays into two sub-arrays by choosing an element from the array to be a pivot. Elements less than the pivot are placed in the left sub-array, and elements that are greater than or equal to the pivot are placed in the right sub-array. It has a worst case time complexity of $O(n^2)$.
- The two quicksorts implemented in this program are iterative. One uses a stack and the other uses a queue.
- The test harness will support the following options:
    - -a: runs all sorting algorithms
    - -b: runs bubble sort
    - -s: runs shell sort
    - -q: runs quicksort that uses stack
    - -Q: runs quicksort that uses queue
    - -r seed: sets random seed; default seed is 13371453
    - -n size: sets the array size; default size is 100
    - -p elements: prints out *elements* number of elements from array; default number of elements to print out is 100

- The test harness will create a random array using the provided seed. The same array will be created for each sorting algorithm by seeding srandom() again with the same seed for each sort.

Pseudocode

Set abstract data type code was provided by Eugene on Piazza

```
set_empty():
        Return 0x00000000

set _member(Set s, x):
        Return s & (1 << (x % SET_CAPACITY)

set_insert(Set s, x):
        Return s | (1<<(x % SET_CAPACITY)

set_remove(Set s, x):
        Return s & ~(1<<(x % SET_CAPACITY))

Set_union(Set s, x):
```

```
        Return s | t

set_intersect(Set s, Set t):
        Return s & t

set_difference(Set s, Set t):
        Return s & ~t

set_complement(Set s):
        Return ~s
```

Python pseudocode was provided in assignment pdf

Bubble.c

```
b_moves=0
b_compares=0

bubble_sort(*A,n):
        Declare int temp
        swapped=1
        While (swapped):
                swapped=0
                For (i=1; i<n; i++):
                        B_compares++
                        If (A[i]<A[i-1]):
                                temp=A[i]
                                A[i]=A[i-1]
                                A[i-1]=temp
                                swapped=1
                                b_moves+=3
                n-=1
```

Python pseudocode was provided in assignment pdf

Shell.c

```
s_moves=0
s_compares=0

shell_sort(*A, n):
        Declare int j
        Declare int temp
        For (k=0;k<sizeof(gaps)/4; k++):
                S_compares++
                j=i
                temp=A[i]
```

```
While (j>=gaps[k] and temp<A[j-gaps[k]]
        A[j]=A[j-gaps[k]]
        j-=gaps[k]
A[j]=temp
s_moves+=2
```

Paper Work

Bubble Sort

	8, 7, 9, 22, 5, 13, 31	n = 6
	7, 8, 9, 5, 13, 22, 31	5
	7, 8, 5, 9, 13, 22, 31	4
	7, 5, 8, 9, 13, 22, 31	3
	5, 7, 8, 9, 13, 22, 31	2
done	5, 7, 8, 9, 13, 22, 31	1

Shell Sort

		Gap
[9, 4, 1, 5, 6, 3]	temp=6	4
[6, 4, 1, 5, 9, 3]	temp=3	2
[6, 3, 1, 5, 9, 4]	temp=1	1
[1, 3, 6, 5, 9, 4]	temp=9	
[1, 3, 6, 5, 9, 4]	temp=5	
[1, 3, 6, 5, 9, 4]	temp=4	
[1, 3, 6, 4, 9, 5]	temp=3	
[1, 3, 6, 4, 9, 5]	temp=6	
[1, 3, 6, 4, 9, 5]	temp=4	
[1, 3, 4, 6, 9, 5]	temp=9	
[1, 3, 4, 6, 9, 5]	temp=5	
[1, 3, 4, 6, 5, 9]	temp=5	
[1, 3, 4, 5, 6, 9]	done	